

TD 6 : Processus, fork, signaux

création de processus, wait • signaux

1 Fork en séquence et en parallèle

Q2 Chacun des fils doit rendre à la terminaison son numéro de création. Le processus main doit afficher la valeur de sortie de chacun des fils et le pid du fils dont il vient de détecter la terminaison.

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main () {
    int N = 4; // 定义一个变量N, 用于指定循环的次数, 这里设置为4
    std::cout << "main pid=" << getpid() << std::endl; // 打印出主进程的进程ID
    for (int i = 0 ; i < N ; i++ ) { // 开始一个循环, 将会创建N个子进程
        if (fork() == 0) { // fork()创建子进程, 如果返回值为0, 则表示是子进程运行的代码
            // 子进程的代码
            std::cout << "fils " << i << " pid=" << getpid() << " ppid=" << getppid() << std::endl;
            return i; // 子进程返回一个值, 这个值通常用于表示退出状态
        }
    }

    for (int i = 0 ; i < N ; i++ ) { // 主进程等待所有子进程结束
        int status;
        int pid = wait(&status); // wait()会阻塞主进程, 直到一个子进程结束
        // 打印结束的子进程的PID和它的返回状态
        std::cout << "Fin du fils de pid=" << pid << " return " << WEXITSTATUS(status) << std::endl;
    }
    return 0; // 主进程返回0, 正常退出
}
```

wait(&status): 父进程调用wait函数来等待任一子进程结束。wait函数返回结束的子进程的PID, 并通过status参数传出子进程的退出状态。
WIFEXITED(status): 这是一个宏, 用来检查子进程是否正常结束。只有当子进程正常结束时, 父进程才能从status中得到子进程的退出码。
WEXITSTATUS(status): 这是一个宏, 用来从status中提取子进程的退出码。退出码是子进程通过return语句返回的值, 或者是传递给exit函数的参数。
这样, 父进程就可以知道哪个子进程结束了, 以及它返回的退出码是什么, 退出码代表了子进程的创建序号。

Q3 主进程 (父进程) 在创建一系列子进程后, 能够按照子进程的创建顺序等待它们结束。为了达到这个目的, 需要用到waitpid函数的特定功能。
En supposant que les fils ont la même durée d'exécution, vont-ils mourir dans l'ordre de leur création ? Modifiez le programme pour que le main attende la fin des fils dans l'ordre de leur création.

Donc non, ce sont des hypothèses sur l'ordonneur qu'il nous faudrait. On ne sait pas dans quel ordre ils mourront et encore moins dans quel ordre leur terminaison sera notifiée au père.
Il nous faut la version complète du wait, c'est `waitpid` qui permet d'attendre un processus ou groupe de processus particulier.
Le `pid` c'est :
`pid_t waitpid(pid_t _pid, int *stat_loc, int options);`
NB : Cette API sur les `pid` vaut la peine d'être commentée, elle est récurrente dans le standard (e.g. kill) :

- `pid = -1` c'est n'importe quel processus
- `pid > 0` c'est explicitement un `pid` de processus précis
- `pid = 0` n'importe quel, mais de mon `gid` (group id)
- `pid < -1` n'importe quel du groupe de `gid = -pid`, donc on vise un groupe.

Ici on veut attendre un fils particulier, dont il nous faut donc connaître le `pid`. Le père va stocker les `pid` de ses fils rendus par `fork()` dans un tableau ou un vecteur.

```
int main() {
    int N = 4; // 子进程的数量
    std::cout << "main pid=" << getpid() << std::endl;
    int pids[N]; // 用于存储子进程PID的数组

    for (int i = 0; i < N; ++i) {
        int pid = fork(); // 创建子进程
        if (pid == 0) {
            // 子进程执行的代码
            std::cout << "fils " << i << " pid=" << getpid() << " ppid=" << getppid() << std::endl;
            return i; // 子进程返回创建序号
        } else {
            pids[i] = pid; // 父进程中存储子进程的PID
        }
    }

    for (int i = 0; i < N; ++i) {
        int status;
        int pid = waitpid(pids[i], &status, 0); // 父进程等待特定子进程结束
        std::cout << "Fin du fils de pid=" << pid << " return " << WEXITSTATUS(status) << std::endl;
        // 打印子进程的PID和退出状态
    }

    return 0;
}
```

Q4

un programme qui crée N processus à travers une chaîne de créations, c'est à dire qu'il crée un fils qui crée un fils... sur N générations. Le père ne doit se terminer que quand tous ses fils sont terminés.

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int N = 4; // 进程链的长度
    std::cout << "main pid=" << getpid() << std::endl;
    int i = 0;
    for (; i < N; i++) {
        int pid = fork();
        if (pid == 0) { // 子进程
            std::cout << "fils " << i << " pid=" << getpid() << " ppid=" << getppid() << std::endl;
            // 如果不是链中的最后一个子进程, 继续循环创建下一个子进程
        } else {
            break; // 父进程跳出循环, 等待子进程
        }
    }
    if (i != N) { // 如果不是最后一个子进程
        int status;
        int pid = wait(&status); // 等待直接子进程结束
        // 输出子进程结束的信息
        std::cout << "Detection par " << getpid() << " Fin du fils de pid=" << pid
            << " return " << WEXITSTATUS(status) << std::endl;
    }
    return 0;
}
```

fork()调用复制当前进程（父进程）创建一个新的进程（子进程）。

在子进程中，fork()返回0。

在父进程中，fork()返回子进程的PID（非零值）。

如果fork()返回0，说明当前进程是子进程，子进程将打印其信息（进程ID和父进程ID），并检查是否是进程链中的最后一个进程：

如果是，它将返回一个特殊值（通常是-1）表示它是链条中的最后一个进程。

如果不是，它将继续循环创建下一个子进程。

如果fork()返回非零值，说明当前进程是父进程，它将break跳出循环，这样每个父进程都只会创建一个子进程。

每个父进程（除了最初的父进程）将执行wait(&status)等待它的直接子进程结束，并打印子进程的退出状态。

0.2 Fork arbitraires et Arbre des processus

```
int main () {
    int N = 4; // 初始化N为4, 用于控制进程分叉数量
    int i = 0; // 初始化计数器i为0
    std::cout << "pid=" << getpid() << std::endl; // 输出当前进程的PID
    while (i < N) { // 当i小于N时循环
        i++; // i自增
        int j = i; // 将i的值赋给j
        if (fork() == 0) { // 如果fork返回0, 则在子进程内部
            if (i%2==0) { // 如果i是偶数
                N = i - 1; // 将N设置为i减1
                i = 0; // 将i重置为0
            } else { // 如果i是奇数
                N = 0; // 将N设置为0
            }
            std::cout << "pid=" << getpid() << " ppid=" << getppid() << " j=" << j << " N=" << N << std::endl;
        }
    }
    return 0; // 主进程结束
}
```

Handwritten notes for the while loop:

- $i=0, j=1, N=0$
- $i=1, j=2, N=0$
- $i=2, j=3, N=1$
- $i=3, j=4, N=3$

10 processus total avec le main.
N donne le nombre de fils, j varie de 1 à N.
Si j est pair, on engendre j-1 fils.

```
[ythierry@localhost src]$ ./a
pid=26918
pid=26919 ppid=26918 j=1 N=0
pid=26920 ppid=26918 j=2 N=1
pid=26921 ppid=26918 j=3 N=0
pid=26922 ppid=26918 j=4 N=3
```

```
# pere = fils N=1
pid=26923 ppid=26920 j=1 N=0
```

```
# pere = fils N=3
pid=26924 ppid=26922 j=1 N=0
pid=26925 ppid=26922 j=2 N=1
pid=26926 ppid=26922 j=3 N=0
```

```
# pere = fils N=1 du fils N=3
pid=26927 ppid=26925 j=1 N=0
```

On peut faire un dessin arborescent au tableau.

Q5

En comptant le processus initial, combien de processus engendre l'exécution du programme ? Donnez l'arbre des descendance de processus, et l'affichage de chacun d'entre eux.

父进程--->4个子进程 (i=1,2,3,4)

- 1.子进程 (i=1) 没有子子进程 (因为if之后n=0)
 - 2.子进程 (i=2) 有一个子子进程 (因为if之后n=1)
 - 3.子进程 (i=3) 没有子子进程 (因为if之后n=0)
 - 4.子进程 (i=4) 有三个子子进程 (i=1,2,3) (因为if之后n=3)
- ①子子进程 (i=1) 没有子子子进程 (因为if之后n=0)
 - ②子子进程 (i=2) 有一个子子子进程 (因为if之后n=1)
 - ③子子进程 (i=3) 没有子子子进程 (因为if之后n=0)

Q6

父进程在所有子进程都完成它们的执行并退出后才结束。这是通过在父进程中添加一个额外的for循环来实现的，该循环中的wait(nullptr)调用会一直阻塞父进程，直到一个子进程结束

. Modifiez le programme pour que le père ne se termine qu'après que tous les autres processus soient terminés. On n'attendra pas plus de fils qu'on en a crée et on ne limitera pas le parallélisme.

On peut utiliser "wait(nullptr)", N fois vu que c'est le nombre de fils. On le place après la boucle.

Attention il faut <sys/wait.h> en plus de <unistd.h>.

```
for (i=0 ; i < N ; i++) {
    wait(nullptr);
}
return 0;
```

0.4 Tour par tour

On veut construire un programme où deux processus s'exécutent tour à tour. Pour cela on demande de n'utiliser que le mécanisme des signaux. 两个进程应该交替执行，使用信号来控制执行流

Question 7. Ecrire un tel programme : 父进程开始第一轮，但在开始之前等待一秒钟。子进程开始它的轮次时首先等待一个信号。

- Le père commence le premier tour, mais il attend une seconde avant de commencer. Le fils commence son tour par attendre un signal. 每个进程在其轮次中显示带有其pid的消息，并显示当前轮次号。
- Chaque processus quand il a la main affiche un message avec son pid, et le tour auquel il en est. 每个轮次结束时，当前进程发送一个信号给另一个进程，然后暂停等待信号。
- A la fin de chaque tour on passe la main à l'autre processus, pour cela on lui envoie un signal, puis on se suspend en attente de sa réponse.
- Chacun des processus fait M tours d'alternance. 每个进程执行M轮交替。

Conceptuellement : On place un **handler**, par exemple qui affiche simplement le pid du récepteur. Sinon on aura le comportement par défaut, avec SIGINT ce serait mourir; pour e.g. SIGUSR1 ce serait ignorer ce qui n'est pas bon.

signal(SIGUSR1, [(int)])

On signale l'autre avec kill :

kill(other, SIGUSR1)

On attend la signalisation avec :

sigsuspend(SIGUSR1)

c'est tout.

On fork, les deux processus bouclent sur un affichage + passer la main.

alternate.cpp

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include <csignal>

int main () {
    const int NBOUND= 5;
    std::cout << "main pid=" << getpid() << std::endl;
    // pid pere
    pid_t mainpid = getpid();

    // 设置信号集，准备阻塞所有信号除了 SIGINT
    // un masque pour passer a sigsuspend dans les deux processus
    sigset_t setneg; // neg car c'est tout sauf SIGINT
    sigfillset(&setneg); // 初始化信号集，包含所有信号
    sigdelset(&setneg, SIGINT); // 从信号集中删除SIGINT，因为这个信号用于终止程序

    // un handler commun aux deux processus // 设置SIGINT的信号处理函数，输出收到的信号
    signal(SIGINT, [(int sig){ std::cout << "processus " << getpid() << "recu signal "
        << sig << std::endl; }]);

    // 设置一个全新的信号集为setpos，阻塞SIGINT
    // BLOC SUIVANT = REPONSE QUESTION 2 *****
    // proteger le fils du premier signal avant sigsuspend
    sigset_t setpos; // pos car c'est seulement SIGINT
    sigemptyset(&setpos); // 初始化setpos，不包含任何信号
    sigaddset(&setpos, SIGINT); // 向setpos添加SIGINT信号
    // SIG_BLOCK = ajouter au masque actif
    // l'ensemble de signaux a ajouter = setpos
    // l'ancien masque si ca nous interesse, ou nullptr
    sigprocmask(SIG_BLOCK, &setpos, nullptr); // 阻塞SIGINT信号，以便在调用sigsuspend之前不会处理它
    // *****

    // pid fils ou 0
    pid_t pid = fork();

    if (pid == 0) {
        // code fils
        std::cout << "Naissance fils " << getpid() << std::endl;
        for (int i = 0 ; i < NBOUND ; i++) {
            sigsuspend(& setneg); // 等待父进程的信号
            // 线程会挂起等待SIGUSR1，而忽略其他信号，直到SIGINT到来
            std::cout << "fils " << i << " pid=" << getpid() << " ppid=" <<
                getpid()
                << " Tour : " << i << std::endl;

            kill(mainpid, SIGINT); // 发送信号给父进程
        }
        std::cout << "Mort du fils" << std::endl;
        return 0;
    } else {
        // donner du temps au fils pour naitre et atteindre sigsuspend
        // En question 1 : on fait un sleep ici. Supprime en question 2
        // sleep(1);
    }
}
```

```
53 // code pere
54 for (int i = 0 ; i < NBOUND ; i++) { // 输出父进程的PID和当前轮次
55     std::cout << "Pere " << i << " pid=" << getpid()
56         << " Tour : " << i <<
57         std::endl;
58
59     // si le fils n'existe pas encore ici, on perd le premier signal
60     kill(pid, SIGINT); // 发送信号给子进程
61     sigsuspend(& setneg); // 等待子进程的信号
62 }
63 wait(nullptr);
64 std::cout << "Mort du pere" << std::endl;
65 }
66 return 0;
```

Question 8. Si le père n'attend pas avant de démarrer, on constate que parfois le fils reçoit le premier signal AVANT de faire le *sigsuspend*. Quel est l'effet sur le programme de ce comportement ? Proposez une correction utilisant les masques (sigprocmask) pour corriger ce problème.

如果父进程没有开始执行之前等待，有时子进程可能会在执行sigsuspend之前就收到了信号。如果这种情况发生，子进程会在sigsuspend调用时被阻塞，因为它在等待的信号已经被处理了，这会导致程序无法继续执行。

Si le fils reçoit le signal AVANT le sigsuspend (et donc le traite), le sigsuspend deviendra bloquant, le programme entier se bloque quand le fils s'échoue sur sigsuspend.

Donc on invoque sigprocmask, AVANT de faire le fork.

Le masque n'empêche pas du tout de faire sigsuspend, l'appel à sigsuspend revient tout de suite si le signal était reçu et pending.

在fork()调用创建子进程之前，先用sigprocmask阻塞那些我们不想让子进程立即处理的信号。这样，即使这个信号在fork()后立即发送到子进程，它也不会被处理，因为它被阻塞了。然后子进程可以在它准备好接收信号时，通过sigsuspend来解除对该信号的阻塞。

0.5 Signaux, Alarmes

创建N个子进程并在等待T秒后向所有子进程发送SIGINT信号。子进程在收到信号后会打印一条消息表示它们即将结束。父进程需要等待所有子进程结束后才退出，并打印消息“Fin du programme”。

On désire réaliser un programme qui crée N processus fils en parallèle, puis attend T secondes avant d'envoyer un SIGINT à tous ses fils. A leur création, les processus fils se bloquent en attente du signal de leur père ; à la réception de ce signal, ils affichent un message indiquant qu'ils vont se terminer. Le processus père attend que tous ses fils soient finis avant de se terminer en affichant un message "Fin du programme".

使用alarm函数来等待T秒后自动发送SIGALRM信号给父进程，然后父进程在接收到SIGALRM后向所有子进程发送SIGINT信号。每个子进程都会设置一个信号处理函数来响应SIGINT，打印消息并退出。

Question 9. Ecrivez ce programme.

alarm.cpp

```
1  #include <csignal>
2  #include <iostream>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main () {
7      int N = 4;
8      std::cout << "main pid=" << getpid() << std::endl;
9      int pids[N]; // 存储子进程ID的数组
10     for (int i = 0 ; i < N ; i++ ) {
11         int pid = fork();
12         if (pid == 0) {
13             // code fils
14             std::cout << "fils " << i << " pid=" << getpid() << " ppid=" <<
15                 getppid() << std::endl;
16
17             // handler sigint // 子进程设置信号处理函数，用于响应SIGINT信号
18             signal(SIGINT, [](int sig) {
19                 std::cout << "Received SIGINT, quitting pid=" << getpid() << std
20                     ::endl;
21                 exit(0); // 立即退出子进程
22             });
23             // bloquer en attente de SIGINT
24             sigset_t set;
25             sigfillset(&set);
26             sigdelset(&set, SIGINT);
27             sigsuspend(&set); // 挂起子进程，直到收到SIGINT信号
28         } else {
29             pids[i] = pid;
30         }
31     }
32     alarm(3); // 父进程设置定时器，在3秒后发送SIGALRM信号
33     // handler pour sigalrm
34     // on pourrait aussi kill les fils dans le handler directement, mais autant éviter
35     // la sémantique des handlers est assez compliquée comme ça
36     // ici un handler vide (!= SIG_IGN on prend quand même le signal !)
37     signal(SIGALRM, [](int) {}); // 设置SIGALRM的空处理函数，仅用于唤醒sigsuspend
38
39     // bloquer en attente de SIGALRM
40     sigset_t set;
41     sigfillset(&set);
42     sigdelset(&set, SIGALRM); // 从集合中删除SIGALRM信号，以便能够接收
43     sigsuspend(&set); // 挂起父进程，直到收到SIGALRM信号
44
45     // on a été réveillé
46     for (int i = 0; i < N ; i++) {
47         kill(pids[i], SIGINT);
48     } // 当定时器超时后，父进程被唤醒，向所有子进程发送SIGINT信号
49
50     for (int i = 0 ; i < N ; i++ ) {
51         wait(nullptr);
52     }
53
54     return 0;
55 }
```


Question 10. Le père peut-il notifier tous ses fils sans avoir stocké leur pid ? Ecrivez une version qui a ce comportement.

父进程能够通知所有子进程而不需要存储它们的PID

Retour sur l'API par groupes de pid déjà vue au début du TD, oui, il suffit de notifier 0 ça crée un broadcast du signal à tout le groupe.

Par contre on va s'auto kill si on ne fait pas attention ! Donc mettre un handler SIG_IGN pour SIGINT avant le broadcast.

当父进程调用kill(0, SIGINT)时，它会向同一进程组中的所有进程发送SIGINT信号

```
alarm.cpp

1 alarm(3);
2 // handler pour sigalrm
3 // on pourrait aussi kill les fils dans le handler directement, mais autant eviter
4 // la semantique des handlers est assez complique comme ca
5 signal(SIGALRM, [](int) {});
6
7 // bloquer en attente de SIGALRM

sigset_t set;
sigfillset(&set);
sigdelset(&set, SIGALRM);
sigsuspend(&set); 暂停进程，直到收到SIGALRM信号

// on a ete reveille
// se proteger du SIGINT pour ne pas s'auto kill
signal(SIGINT, SIG_IGN); // 为了防止父进程在发送SIGINT信号时自己也被
// ok on broadcast      终止，先将SIGINT信号的处理方式设置为忽略
kill(0, SIGINT); // 向当前进程组中的所有进程发送SIGINT信号，包括父进程和所有子进程

for (int i = 0 ; i < N ; i++ ) {
    wait(nullptr);
}
```

TME 6 : Processus, Signaux
Objectifs pédagogiques :
• fork, wait
• signaux

1.2 Encore un Arbre des processus

```
forkexo2.cpp

#include <iostream>
#include <unistd.h>

int main () {
    const int N = 3;
    std::cout << "main pid=" << getpid() << std::endl;

    for (int i=1, j=N; i<=N && j==N && fork()==0 ; i++ ) {
        std::cout << " i:j " << i << " : " << j << std::endl;
        for (int k=1; k<=1 && j==N ; k++) {
            if ( fork() == 0 ) {
                j=0;
                std::cout << " k:j " << k << " : " << j << std::endl;
            }
        }
    }

    return 0;
}
```

Question 1. En comptant le processus initial, combien de processus engendre l'execution du programme ? Donnez l'arbre des descendances de processus, et l'affichage de chacun d'entre eux.

10 processus en comptant le main.

[ythierry@localhost src]\$./a.out
main pid=32503

pid=32504 ppid=32503 i:j 1:3

pid=32505 ppid=32504 k:j 1:0
pid=32506 ppid=32504 i:j 2:3

pid=32507 ppid=32506 k:j 1:0
pid=32508 ppid=32506 k:j 2:0
pid=32509 ppid=32506 i:j 3:3

pid=32511 ppid=32509 k:j 1:0
pid=32512 ppid=32509 k:j 2:0
pid=32513 ppid=32509 k:j 3:0

Question 11. La terminaison (exit) d'un processus fils engendre un signal *SIGCHLD* à destination du père, le comportement par défaut étant de l'ignorer. Le père peut-il s'appuyer sur ce mécanisme plutôt que sur *wait* pour attendre la terminaison de ses fils ?

Non, on ne peut pas compter les signaux recus, on a toutes les chances d'en perdre en fait, pas de façon de compter fiable.

Les signaux "pending" (i.e. délivrés par l'OS au processus, mais qu'il n'a pas encore traité, par exemple parce qu'il n'est pas élu) sont stockés dans un bitset, on ajoute les signaux recus avec un OR bit à bit.

Donc on perd des signaux *SIGCHLD* potentiellement, pas de façon de contrôler ou remédier, à part utiliser *wait*.

不应该依赖接收到的SIGCHLD信号计数来判断子进程何时结束，因为信号可能会丢失，这不是一个可靠的计数方式。当多个信号几乎同时到达时，操作系统可能会合并这些信号，只传递一个信号给进程，这就导致了信号的丢失。信号的"pending"状态（即，已经交付给进程但还未处理的信号）是由操作系统维护的，通常存储在一个位集（bitset）中，如果在处理之前有相同的信号再次发生，新的信号不会增加pending信号的数量，只会设置相应的位。

Question 2. Modifiez le programme pour que le père ne se termine qu'après que tous les autres processus soient terminés. On n'attendra pas plus de fils qu'on en a crée et on ne limitera pas le parallélisme. 父进程必须在所有其他子进程结束后才能结束。不需要等待那些已经创建并且已经结束的子进程，同时也不应该限制并发性

NB: On recommande d'ajouter et de maintenir à jour un **compteur** du nombre de fils engendrés par un processus, et de faire le **wait** dans une boucle séparée après le corps du programme fourni.

添加并更新一个计数器来跟踪每个进程创建的子进程数量，并在程序主体提供的之后，用一个单独的循环来等待（wait）每个子进程结束

Ok c'est plus dur car il y a deux fork. Le premier est en plus dans une condition, il faut tester pour être sur de devoir faire un wait. Et il faut bien lire les conditions et comprendre le code pour ecrire les bons wait, bref galère.

Comme la question exige en plus de ne pas limiter le parallélisme, le plus simple reste de compter les fils créés et de wait à la fin. Attention on incrémente si on est le pere dans le fork, mais SINON on remet à zéro le compteur (sinon on comptera ses frères aînés comme des fils).

main pid=2472	pid=2480 ppid=2479 k:j 1:0 Fin du processus 2480
pid=2474 ppid=2472 i:j 1:3	pid=2484 ppid=2479 k:j 2:0 Fin du processus 2484
pid=2476 ppid=2474 i:j 2:3	pid=2485 ppid=2479 k:j 3:0 Fin du processus 2485
pid=2475 ppid=2474 k:j 1:0 Fin du processus 2475	detection par 2479 de fin du fils de pid=2 detection par 2479 de fin du fils de pid=2 detection par 2479 de fin du fils de pid=2
pid=2477 ppid=2476 k:j 1:0 Fin du processus 2477	Fin du processus 2479 detection par 2476 de fin du fils de pid=2
pid=2478 ppid=2476 k:j 2:0 Fin du processus 2478	Fin du processus 2476 detection par 2474 de fin du fils de pid=2
detection par 2474 de fin du fils de pid=2475	Fin du processus 2474 detection par 2472 de fin du fils de pid=2
pid=2479 ppid=2476 i:j 3:3	Fin du processus 2472
detection par 2476 de fin du fils de pid=2477 detection par 2476 de fin du fils de pid=2478	

```
forkexo2.cpp

#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main () {
    const int N = 3;
    std::cout << "main pid=" << getpid() << std::endl;

    int nbfiles = 0 ;
    for (int i=1, j=N ; i<=N && j==N ; i++ ) {
        if (fork() !=0) {
            nbfiles++;
            break;
        } else {
            nbfiles = 0;
        }

        std::cout << "pid=" << getpid() << " ppid=" << getppid()
            << " i:j " << i << " : " << j << std::endl;

        for (int k=1; k<=1 && j==N ; k++) {
            if ( fork() == 0 ) {
                nbfiles = 0;
                j=0;
                std::cout << "pid=" << getpid() << " ppid=" << getppid()
                    << " k:j " << k << " : " << j << std::endl;
            } else {
                nbfiles++;
            }
        }
    }

    for (int i=0 ; i < nbfiles ; i++) {
        int status;
        int pid = wait(&status); // 等待子进程结束并获取子进程ID
        std::cout << "detection par " << getpid() << " de fin du fils de pid=" <<
            pid << std::endl; // 打印检测到的子进程结束信息
    }

    std::cout << "Fin du processus " << getpid() << std::endl;
    return 0; // 打印父进程结束信息
}
```

Question 5. Comment assurer que les deux processus qui s'affrontent utilisent une graine aléatoire différente ? 随机数函数 (如rand()) 通常需要一个“种子”来开始生成随机数序列。如果两个进程使用相同的种子，它们将生成相同的随机数序列。每个进程有一个独一无二的ID，使用getpid()的返回值作为种子，可以保证每个进程得到的种子是唯一的，从而产生不同的随机数序列。

On pose une graine avec srand, basée sur time(0) ou sur getpid.

On propose de modifier la défense de Luke (le fils) pour qu'il affiche les coups parés. On propose procéder de la manière suivante :

- positionner un handler qui affiche "coup paré" avec sigaction
- masquer les signaux avec sigprocmask ;
- s'endormir pendant une durée aléatoire avec randsleep ;
- invoquer sigsuspend pour tester si une attaque a eu lieu, et donc afficher le message "coup paré" si le signal a été reçu.

Question 6. Le combat est-il encore équitable ? Expliquez pourquoi.

Non, on attend le signal = on pare à tous les coups

使用sigsuspend, Luke进程在休眠期间暂停执行直到收到信号。如果攻击发生 (即vador发送了信号), 它将被捕捉并处理, 显示“招架”信息。由于sigsuspend会等待一个信号的发生, 这意味着Luke总能在攻击发生时醒来并招架, 从而不会受到任何攻击的影响。这样的机制使得Luke总能成功防御每次攻击, 因此战斗变得不公平, 优势在于总是处于防御状态的Luke。

nanosleep函数允许程序暂停执行一段指定的时间。如果nanosleep因为接收到信号而被打断 (返回EINTR错误), 它会将剩余的睡眠时间写入remain结构体。这样, 可以通过一个循环来确保程序完成整个指定的睡眠时长, 即使在睡眠期间收到了信号。这对于攻击阶段非常重要, 因为即使在收到攻击信号时, 我们也希望进程能够完成其随机时长的睡眠

combat.cpp

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include "rsleep.h"

// points de vie, variable globale, dupliquee au moment du fork
int PV = 8; 在fork时复制

// quand on est en attaque : decremente les PV
void handler (int sig) {
    if (sig == SIGUSR1) { // 如果收到的是SIGUSR1信号
        PV--;
        printf("Attaque reçue par %d; PV restants %d\n",getpid(),PV);
        if (PV == 0) {
            printf("Plus de vie pour %d; mort du processus.\n",getpid());
            exit(1);
        }
    }
}

void attaque (pid_t adversaire) {
    signal(SIGUSR1,handler); // 设置SIGUSR1信号的处理函数为handler
    if (kill(adversaire,SIGUSR1) < 0) { // 向对手发送SIGUSR1信号
        printf("Detection de Mort de l'adversaire de pid=%d \n",adversaire);
        exit(0); // 如果发送失败, 打印对手死亡信息
    }
    randsleep(); // 随机睡眠一段时间
}

void defense () {
```

```
    signal(SIGUSR1,SIG_IGN); // 忽略SIGUSR1信号

    randsleep();
}

// 交替进行防御和攻击的函数
void combat(pid_t adversaire) {
    while (1) { // 无限循环
        defense(); // 先进行防御
        attaque(adversaire); // 再进行攻击
    }
}

int main() {
    pid_t pere = getpid();
    pid_t pid = fork();
    srand(pid); // 设置随机种子, 以便产生随机数
    if (pid == 0) {
        combat(pere); // 子进程与父进程战斗
    } else {
        combat(pid);
    }
    return 0;
}
```

```
#include <time.h>

void randsleep() {
    int r = rand();
    double ratio = (double)r / (double) RAND_MAX;
    struct timespec tosleep;
    tosleep.tv_sec = 0;
    // 300 millions de ns = 0.3 secondes
    tosleep.tv_nsec = 300000000 + ratio*700000000;
    struct timespec remain;
    while ( nanosleep(&tosleep, &remain) != 0) {
        tosleep = remain;
    }
}
```

La doc du man nous révèle que se faire signaler cause un EINTR, et écrit le temps restant dans remain. On a bien l'intention de se faire signaler pendant qu'on dort, du moins en attaque, donc il faut faire cette boucle.

de ce fil, alors que l'attente est comme dans la question précédente.

Question 8. Toujours sans utiliser *waitpid*, mais à plutôt l'aide de signaux, implanter ce comportement. Attention à bien désarmer une alarme temporisée si le fils meurt avant le temps imparti. Indice : On pourra se placer en attente des deux signaux : SIGCHLD (un fils est mort) et SIGALRM (temporisateur). 处理子进程的结束和超时 (定时器)

如果子进程的睡眠时间超过父进程等待的时间（在这个例子中是3秒），父进程会因为超时而停止等待，并且wait_till_pid函数将返回-1。如果子进程在3秒内结束，wait_till_pid函数将返回子进程的PID

pseudowait.cpp

```
1 #include <iostream>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5
6 // V1
7 pid_t wait_till_pid(pid_t pid) {
8     while (true) {
9         pid_t p = wait(nullptr); // 等待任何子进程结束
10        if (p == -1 || p == pid) { // 如果等待出错或等待的子进程结束
11            return p;
12        }
13    }
14 }
15
16 // V2 : signal 束, 函数返回子进程的PID; 如果超时, 则返回0; 如果出现错误 (例如, 子进程不存在), 则返回-1。
17 pid_t wait_till_pid(pid_t pid, int sec) {
18     static bool timeout;
19     timeout = false; // 设置SIGALRM的处理函数, 当超时发生时将timeout标记为true
20     signal(SIGALRM, [](int) { std::cout << "received SIGALRM" << std::endl; timeout =
21         true; }); // 设置SIGCHLD的处理函数, 当子进程结束时调用
22     signal(SIGCHLD, [](int) { std::cout << "received SIGCHLD" << std::endl; });
23     alarm(sec); // 设置一个sec秒后发生的超时信号
24
25     sigset_t set;
26     sigfillset(&set);
27     sigdelset(&set, SIGALRM);
28     sigdelset(&set, SIGCHLD);
29
30     while (true) {
31         std::cout << "waiting..." << std::endl;
32         sigsuspend(&set); // 暂停进程, 直到接收到集合中未被阻塞的信号
33         if (timeout) { // 如果发生超时
34             std::cout << "Alarm wins" << std::endl;
35             return -1; // 返回-1表示超时
36         } else {
37             pid_t p = wait(nullptr); // 检查子进程是否结束
38             std::cout << "wait answered " << p << std::endl;
39             if (p == pid) {
40                 alarm(0); // 如果是指定的子进程结束, 取消超时
41             }
42             if (p == -1 || p == pid) {
43                 return p;
44             }
45         }
46     }
47 }
48
49 int main() {
50     pid_t pid = fork();
51     if (pid == 0) {
52         // si on met plus que le timeout il doit mourir
53         sleep(5);
54     } else {
55         signal(SIGINT, [](int) {}); // 设置SIGINT信号的处理函数, 此处为空操作
56         pid_t res = wait_till_pid(pid, 3); // 父进程等待子进程最多3秒
57         std::cout << "wait gave :" << res << std::endl;
58     }
59 }
```